Python: Visualization
Compilation: Yogesh Haribhau Kulkarni

Visualization

Bokeh

(Ref: Creating Interactive Visualizations with Bokeh - Fonnesbeck)

What is Bokeh?

- Bokeh is a Python package for creating interactive, browser-based visualizations
- High-level commands for data binding, transformation, interaction
- Low-level power to deeply customize

How does Bokeh work?

- Bokeh writes to a custom-built HTML5 Canvas library, which affords it high performance.
- This allows it to integrate with other web tools, such as Google Maps.
- Bokeh plots are based on visual elements called glyphs that are bound to data objects.

Intuition

Think of a Bokeh plot as a small web app, not a static picture: matplotlib bakes pixels into an image, while Bokeh ships JSON describing your data plus JavaScript that draws and updates it in the browser – that's what makes panning, zooming, and hovering possible without re-running Python.

A Simple Example

Every Bokeh plot starts the same way: create a `figure`, add glyphs to it with method calls, then `show()` it.

```
from bokeh.plotting import figure, show
import numpy as np

x = np.linspace(-6, 6, 100)
y = np.random.normal(0.3*x, 1)

p = figure(width=500, height=500)
p.scatter(x, y, color="red", marker="circle")
show(p)
```

Statistical Plots

```
from scipy.stats import expon
theta = 1
measured = np.random.exponential(theta, 1000)
hist, edges = np.histogram(measured, density=True,
     bins=50)

p = figure(title="Exponential Distribution
     (theta=1)", tools="save",
           background_fill_color="#E8DDCB")
p.quad(top=hist, bottom=0, left=edges[:-1],
     right=edges[1:],
       fill_color="#036564", line_color="#033649")
```

Statistical Plots

Add lines showing the form of the probability distribution function (PDF) and cumulative distribution function (CDF), on the same figure `p` created on the previous slide.

```
x = np.linspace(0, 10, 1000)
p.line(x, expon.pdf(x, scale=1),
     line_color="#D95B43",
       line_width=8, alpha=0.7, legend_label="PDF")
p.line(x, expon.cdf(x, scale=1),
     line_color="white",
       line_width=2, alpha=0.7, legend_label="CDF")
p.legend.location = "top_right"

show(p)
```

Bar Plot Example

- Bokeh's core display model relies on composing graphical primitives which are bound to data series.
- We first need to reshape the data, by grouping it according to the year of the car, and then by the country of origin (here, USA or Japan).

```
grouped = autompg.groupby("yr")
mpg = grouped["mpg"]
mpg_avg = mpg.mean()
mpg_std = mpg.std()
years = np.asarray(list(grouped.groups.keys()))
american = autompg[autompg["origin"]==1]
japanese = autompg[autompg["origin"]==3]

american.head(10)
```

Bar Plot Example

	mpg	cyl	displ	hp	weight	accel	yr	origin	name
0	18	8	307	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15	8	350	165	3693	11.5	70	1	buick skylark 320
2	18	8	318	150	3436	11.0	70	1	plymouth satellite
3	16	8	304	150	3433	12.0	70	1	amc rebel sst
4	17	8	302	140	3449	10.5	70	1	ford torino
5	15	8	429	198	4341	10.0	70	1	ford galaxie 500
6	14	8	454	220	4354	9.0	70	1	chevrolet impala
7	14	8	440	215	4312	8.5	70	1	plymouth fury iii
8	14	8	455	225	4425	10.0	70	1	pontiac catalina
9	15	8	390	190	3850	8.5	70	1	amc ambassador dpl

Bar Plot Example

- For each year, we want to plot the distribution of MPG within that year.
- As a guide, we will include a box that represents the mean efficiency, plus and minus one standard deviation.
- We will make these boxes partly transparent.

```
p = figure(title='Automobile mileage by year and
     country')
p.quad(left=years-0.4, right=years+0.4,
       bottom=mpg_avg-mpg_std,
     top=mpg_avg+mpg_std, fill_alpha=0.4)
```

Bar Plot Example

We overplot the actual data points on the same figure `p`, using contrasting symbols for American and Japanese cars.

```
# Add Japanese cars as circles
p.scatter(x=np.asarray(japanese["yr"]),
          y=np.asarray(japanese["mpg"]),
     marker="circle",
          size=8, alpha=0.4, line_color="red",
     fill_color=None, line_width=2)

# Add American cars as triangles
p.scatter(x=np.asarray(american["yr"]),
          y=np.asarray(american["mpg"]),
     marker="triangle",
```

```
        size=8, alpha=0.4, line_color="blue",
    fill_color=None, line_width=2)
```

Bar Plot Example

We can add axis labels by binding them to the axis_label attribute of each axis.

```
p.xaxis.axis_label = 'Year'
p.yaxis.axis_label = 'MPG'

show(p)
```

Linked Brushing

To link plots together at a data level, we can explicitly wrap the data in a ColumnDataSource. This allows us to reference columns by name.

```
from bokeh.models import ColumnDataSource
from bokeh.layouts import gridplot

source = ColumnDataSource(autompg.to_dict("list"))

plot_config = dict(width=300, height=300,

    tools="pan,wheel_zoom,box_zoom,box_select")

p1 = figure(title="MPG by Year", **plot_config)
p1.scatter("yr", "mpg", color="blue",
    source=source, marker="circle")

p2 = figure(title="HP vs. Displacement",
    **plot_config)
p2.scatter("hp", "displ", color="green",
    source=source, marker="circle")

p3 = figure(title="MPG vs. Displacement",
    **plot_config)
p3.scatter("mpg", "displ", size="cyl",
    line_color="red",
          fill_color=None, source=source,
    marker="circle")

show(gridplot([[p1, p2, p3]]))
```

Visualization of US unemployment rates

- Our first example of an interactive chart involves generating a heat map of US unemployment rates by month and year.
- This plot will be made interactive by invoking a HoverTool that displays information as the pointer hovers over any cell within the plot.

Visualization of US unemployment rates

First, we import the data with Pandas and manipulate it as needed.

```
import pandas as pd
from bokeh.models import HoverTool
from bokeh.sampledata.unemployment1948 import data

data['Year'] = [str(x) for x in data['Year']]
years = list(data['Year'])
months =
     ["Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep",
data = data.set_index('Year')
```

Visualization of US unemployment rates

Specify a color map

```
colors = [
    "#75968f", "#a5bab7", "#c9d9d3", "#e2e2e2",
     "#dfccce",
    "#ddb7b1", "#cc7878", "#933b41", "#550b1d"
]

month = []
year = []
color = []
rate = []
for y in years:
    for m in months:
        month.append(m)
        year.append(y)
        monthly_rate = data[m][y]
        rate.append(monthly_rate)

     color.append(colors[min(int(monthly_rate)-2,
     8)])
```

Visualization of US unemployment rates

Create a ColumnDataSource with columns: month, year, color, rate

```
source = ColumnDataSource(
    data=dict(
        month=month,
        year=year,
        color=color,
        rate=rate,
    )
)
```

Visualization of US unemployment rates

- x_range is years, y_range is months (reversed)
- fill color for the rectangles is the 'color' field
- line_color for the rectangles is None
- tools are the built-in save and hover tools
- add a nice title, and set the width and height

```
p = figure(x_range=years,
     y_range=list(reversed(months)),
           x_axis_location="above", width=900,
     height=400,
           title="US Unemployment (1948 - 2013)",
           tools="save,hover")

p.rect(x='year', y='month', width=0.95,
     height=0.95, source=source,
       fill_color='color', line_color=None)
```

Visualization of US unemployment rates

Style the plot, including:

- remove the axis and grid lines
- remove the major ticks
- make the tick labels smaller
- set the x-axis orientation to vertical, or angled

```
p.grid.grid_line_color = None
p.axis.axis_line_color = None
p.axis.major_tick_line_color = None
p.axis.major_label_text_font_size = "5pt"
p.axis.major_label_standoff = 0
p.xaxis.major_label_orientation = np.pi/3
```

Visualization of US unemployment rates

Configure the hover tool to display the month, year and rate

```
hover = p.select(dict(type=HoverTool))[0]
hover.tooltips = [
    ('date', '@month @year'),
    ('rate', '@rate'),
]
show(p)
```

Intuition

`p.select()` searches the figure for the tool/glyph objects matching a type or name, since Bokeh plots are really trees of Python objects – you configure them by finding and mutating the right node, not by re-issuing a plotting command.

Quick Check: Core Bokeh

Think About It

A matplotlib figure is a static image – once it's drawn, interacting with it in Python again requires re-running the plotting code. Why can a Bokeh figure support panning, zooming, and hover tooltips *without* calling back into your Python process?

Answer

Bokeh doesn't render a picture – it serializes the figure (data via `ColumnDataSource`, glyphs, tools) into JSON and ships a JavaScript library (BokehJS) that runs entirely in the browser. Pan/zoom/hover are handled client-side against that JSON; your Python code has already finished running by the time you interact with the plot.

High-level Plots

- The examples so far have been relatively low-level, in that individual elements of plots need to be specified by hand.
- Bokeh once shipped a `bokeh.charts` module with ready-made chart types (`Bar`, `BoxPlot`, `HeatMap`, `Histogram`, `Scatter`, `Timeseries`) that made smart layout decisions automatically.
- `bokeh.charts` was removed from Bokeh years ago – there is no drop-in high-level charting module anymore, so the same results are built directly with `bokeh.plotting`, just as we've been doing.

High-level Plots

To illustrate, let's create some random data and display it as a histogram, built with the same `figure`/`quad` pattern used earlier for the exponential distribution.

```
normal = np.random.standard_normal(1000)
hist, edges = np.histogram(normal,
     bins=int(np.sqrt(len(normal))), density=True)

p = figure(title="Histogram",
     x_axis_label="value",
     y_axis_label="frequency",
           width=600, height=300)
p.quad(top=hist, bottom=0, left=edges[:-1],
     right=edges[1:],
       fill_color="navy", line_color="white",
     alpha=0.6)
show(p)
```

Scatter Plot

```
from bokeh.sampledata.iris import flowers

colors = {"setosa": "navy", "versicolor": "olive",
     "virginica": "firebrick"}

p = figure(title="Iris dataset",
     x_axis_label="petal_length",
           y_axis_label="petal_width", width=600,
     height=400)
for species, color in colors.items():
     subset = flowers[flowers.species == species]
     p.scatter(subset["petal_length"],
     subset["petal_width"],
               color=color, marker="circle",
     legend_label=species)

p.legend.location = "top_left"
show(p)
```

Working with other plotting libraries

- Bokeh used to ship a `bokeh.mpl` compatibility layer that could convert an existing Matplotlib/Seaborn figure into an interactive Bokeh one – that layer has since been removed.
- There is no automatic conversion today: if you want a Bokeh-interactive version of a plot, you build it directly with `bokeh.plotting`, as below. Matplotlib and Seaborn remain perfectly usable side by side with Bokeh, just as independent, separate figures.

```
titanic = pd.read_csv(

     'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')
titanic = titanic.dropna(subset=['age'])
```

Working with other plotting libraries

A native Bokeh equivalent of a per-class age distribution: overlay one semi-transparent histogram per passenger class.

```
p = figure(title="Titanic passenger age
     distribution by class",
           x_axis_label="age",
     y_axis_label="density", width=600, height=350)
class_colors = {1: "navy", 2: "olive", 3:
     "firebrick"}

for pclass, color in class_colors.items():
     ages = titanic.loc[titanic["pclass"] ==
     pclass, "age"]
     hist, edges = np.histogram(ages, bins=20,
      density=True)
     p.quad(top=hist, bottom=0, left=edges[:-1],
      right=edges[1:],
            fill_color=color, alpha=0.4,
      legend_label=f"Class {pclass}")

show(p)
```

Working with other plotting libraries

Seaborn and Matplotlib still work exactly as before – they simply produce their own separate figure, in their own window/output, instead of feeding into Bokeh.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.violinplot(data=titanic, x="pclass", y="age")
plt.title("Distribution of age by passenger class")
plt.show()
```

Generating and displaying images

- Mandelbrot set images are made by sampling complex numbers and determining for each whether the result tends towards infinity when a particular mathematical operation is iterated on it.
- First, functions for generating the Mandelbrot set image. They create a 2D array of numbers, over which a color map can be displayed.

Generating and displaying images

```
def mandel(x, y, max_iters):
     """
     Given the real and imaginary parts of a
      complex number,
     determine if it is a candidate for membership
      in the Mandelbrot
     set given a fixed number of iterations.
     """
     c = complex(x, y)
     z = 0.0j
     for i in range(max_iters):
          z = z*z + c
          if (z.real*z.real + z.imag*z.imag) >= 4:
               return i
     return max_iters
```

Generating and displaying images

```
def create_fractal(min_x, max_x, min_y, max_y,
     image, iters):
    height = image.shape[0]
    width = image.shape[1]

    pixel_size_x = (max_x - min_x) / width
    pixel_size_y = (max_y - min_y) / height

    for x in range(width):
        real = min_x + x * pixel_size_x
        for y in range(height):
            imag = min_y + y * pixel_size_y
            color = mandel(real, imag, iters)
            image[y, x] = color
```

Generating and displaying images

Define the bounding coordinates to generate the Mandelbrot image, then create a scalar image (2D array of numbers)

```
min_x = -2.0
max_x = 1.0
min_y = -1.0
max_y = 1.0

img = np.zeros((1024, 1536), dtype = np.uint8)
create_fractal(min_x, max_x, min_y, max_y, img, 20)
```

Generating and displaying images

Use the image renderer to display the Mandelbrot image, colormapped with the "Spectral11" palette. The renderer can display many images at once, so it takes lists of images, coordinates, and palettes.

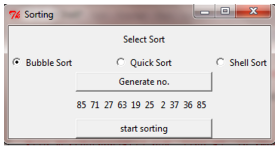
```
p = figure(title="Mandelbrot", x_range=(min_x,
     max_x), y_range=(min_y, max_y),

     tools="pan,wheel_zoom,box_zoom,reset,save",
           width=900, height=600)
p.image(image=[img], x=[min_x], y=[min_y],
        dw=[max_x-min_x], dh=[max_y-min_y],
     palette="Spectral11")
show(p)
```

Tkinter

(Reference: CS2316, Fall 2016
http://cs2316.gatech.edu/slides/tkinter.pptx)

GUI: Graphical User Interface

- A way for people to interact with computer programs
- Uses windows, icons and menus
- Can be manipulated by a mouse and keyboard



GUIs: Step by Step

- Design the GUI
- Create all the buttons, labels, entries, etc. (widgets) And specify where they go in a window
- Write functions/methods to do what needs to be done
- Connect the widgets to the functions/methods
- Start the main loop

Tkinter

- Tkinter is the de facto Python standard GUI library, it comes with your installation of Python and doesn't need to be installed separately
- We will use the Tkinter GUI toolkit to create Python GUIs https://docs.python.org/3/library/tk.html

```
from tkinter import *
```

Intuition

`win.mainloop()` hands control over to Tkinter's event loop – your script effectively pauses there, waiting and reacting to clicks/keystrokes as they happen, instead of running top-to-bottom like a normal script. This is why widgets are created and `.pack()`'d *before* `mainloop()`: nothing shows up on screen until the loop is running.

Some simple tkinter widgets

This code creates a basic GUI with a Label and a Button and places them in a window using the Pack layout manager. The button has no "command" so it has no function when clicked.

- Label: Just displays text, not clickable
- Button: Can be linked to a function/method which is called on click
- Entry: Can allow the user to enter text, Can also be used to display text, Has 3 possible states: normal, readonly, disabled

```
win = Tk()
win.title("My GUI")
l = Label(win,
     text="Hello,
     world")
b = Button(win,
     text="Click me!")
l.pack()
b.pack()

win.mainloop()
```

Some simple tkinter widgets

- Now the clicked function will be called every time the button is clicked
- If we defined the clicked function below the GUI code, the line creating the Button would throw a NameError
- We can avoid this problem by writing a class for our GUI

```
def clicked():
    print("Button
     clicked!")

win = Tk()
win.title("My GUI")
l = Label(win,
     text="Hello,
     world")
b = Button(win,
     text="Click me!",
     command=clicked)
l.pack()
b.pack()

win.mainloop()
```

Encapsulating a GUI into an Object

- The __init__ method takes in a Tk window, creates GUI widgets, and packs them
- Command is now self.clicked because clicked is a method in the class

```
class MyGUI:

  def __init__(self,
     win):
    self.l = Label(win,
     text="Hello,
     world!")
    self.b = Button(win,
     text="Click me!",
     command=self.clicked)
    self.l.pack()
    self.b.pack()

  def clicked(self):
    print("Button
     clicked!")
```

Encapsulating a GUI into an Object

- In the main method, we create a Tk window, give it a title, and pass it as the win argument of a new GUI object

```
def main(args):
    win = Tk()
    win.title("My
     GUI")
    MyGUI(win)
    win.mainloop()

if __name__ ==
     '__main__':
    import sys
    main(sys.argv)
```

GUI layout managers

Pack

- Positions widgets relative to each other
- Simple to use
- Limited layout possibilities compared to grid

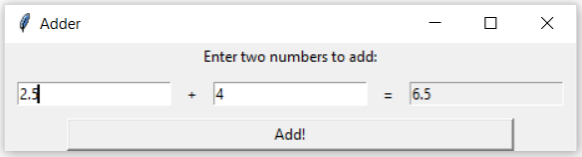
Grid

- Allows you to place widgets in rows and columns
- More complicated than pack but good for GUIs where a particular arrangement is desired

Don't use grid and pack in the same container!

Exercise: Adder

- Create a GUI with two NORMAL entries and one READONLY entry
- There should a button that, when clicked, adds the numbers entered in the first two entries and displays the sum in the third entry
- If the calculation can't be performed due to invalid input, display a message box with an error message.



Frames

- Container that can be placed within a window
- Holds other widgets
- Each frame has its own grid layout, independent of the main window
- You can use a different layout manager for each container

Example: Frames (use within GUI class)

```
f1 = Frame(win, height=100, width=100, bd=1,
     relief=SUNKEN)
f2 = Frame(win, height=100, width=100, bd=1,
     relief=SUNKEN)

f1.grid(row=0, column=0, sticky=EW)
f2.grid(row=0, column=1, sticky=EW)

# self.l1 will be used for another method later
self.l1 = Label(f1, text="One", height=5, width=5)
self.l1.grid(row=0, column=0)
Label(f1, text="Two", height=5,
     width=5).grid(row=0, column=1)
Label(f2, text="Three", height=5,
     width=5).pack(side=LEFT)
Label(f2, text="Four", height=5,
     width=5).pack(side=LEFT)
```

Radiobuttons

- Allows the user to choose from one or more options
- Can be linked to other Radiobuttons with StringVars and IntVars
- If configured correctly, only one Radiobuttons from a group can be selected at at time

StringVars and IntVars

- Two variable classes included in Tkinter
- Have .set() method to set their value e.g. my_string_var.set("Hello")
- StringVars have string value, IntVars have integer value
- Have .get() method to get their value e.g. val = my_string_var.get()
- Can be passed in as "variable" argument when creating a Radiobutton, or "textvariable" argument when creating an Entry

Intuition

A plain Python variable is just a value – assigning to it doesn't touch the screen. A `StringVar/IntVar` is a small wrapper object that Tkinter watches: link it to a widget via `variable/textvariable`, and the widget updates automatically whenever `.set()` changes the wrapper, in either direction.

Exercise

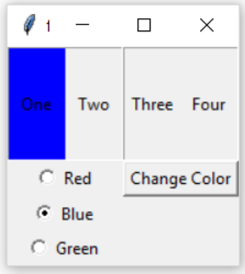
- Revisit our Adder example from the last class
- Instead of using Entry.insert and Entry.delete, use a StringVar to change the text of the result Entry
- See what happens if you don't set the Entry state to "normal" before changing the text!

Adding Radiobuttons to our Frame example

```
self.color = StringVar()

rb1 = Radiobutton(win, text="Red",
     variable=self.color, value="red")
rb2 = Radiobutton(win, text="Blue",
     variable=self.color, value="blue")
rb3 = Radiobutton(win, text="Green",
     variable=self.color, value="green")

self.color.set("unknown") # deselects all
     radiobuttons linked to self.color

rb1.grid(row=1, column=0)
rb2.grid(row=2, column=0)
rb3.grid(row=3, column=0)

Button(win, text="Change Color",
     command=self.change_color).grid(row=1,
     column=1)
```

Adding Radiobuttons to our Frame example

```
def
     change_color(self):
  new_color =
     self.color.get()
  if new_color ==
     "unknown":
    pass
  else:

     self.l1.config(bg=new_
```



Additional Exercise

- Add another set of Radiobuttons to the GUI. Separate from those linked to self.change_color
- There should be four Radiobuttons, each linked to one of the labels
- Create an IntVar to use as the variable for these Radiobuttons
- The user's selection should determine which Label will change color when the button is clicked.

Root Window vs. Toplevels

- The root window is your main window (Tk)
- A GUI application should have only one root window
- Any additional windows should be Toplevels
- Hide windows with .withdraw()
- Show windows with .deiconify()
- Delete windows with .destroy() (they will be gone forever!)
- If you destroy the root window, all Toplevels will also be destroyed

Adding multiple windows to our example

```
# in __init__ method, add line: self.root = win

def new_window(self):
  self.root.withdraw()
  self.new = Toplevel()
  Label(self.new, width=30, height=10, text="I'm a
     new window!").pack()
  Button(self.new, text="Close window",
     command=self.close_new).pack()

def close_new(self):
  self.new.destroy() # destroy new window
  self.root.deiconify() # show root window
```

Changing the GUI appearance

- GUI windows and widgets have many optional parameters to change their appearance
- bg: background color
- fg: foreground color (for Labels, this is the font color)
- bd: border width
- border: border type (RAISED, SUNKEN, etc)
- For color codes: http://htmlcolorcodes.com/
- For more options, check the documentation or online tutorials

Quick Check: Tkinter

Think About It

The Adder exercise wraps its GUI in a class (`MyGUI`) instead of writing the widget-creation code directly at module level, the way the very first Label/Button example did. What breaks if you don't do that once a callback like `clicked()` needs to read or update *other* widgets?

Answer

A bare function callback only has access to whatever it can see via globals or closures – referencing widgets created later in the same script raises a `NameError`, and referencing several widgets from many callbacks gets messy fast. Wrapping everything in a class turns each widget into `self.widget`, so any method (bound via `command=self.clicked`) can freely read and update every other widget through `self`, regardless of creation order.

Copyleft © Send suggestions to yogeshkulkarni@yahoo.com